

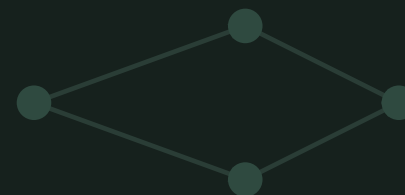


A workflow management tool for reproducible and scalable data analyses

Dr. Jess Grembi

PSU One Health Microbiome Center · KickStart Workshop 2026 · Day 3

Slides designed with help from Claude (Anthropic)



Outline



Why use a workflow management tool



Today's objectives



Overview of Snakemake

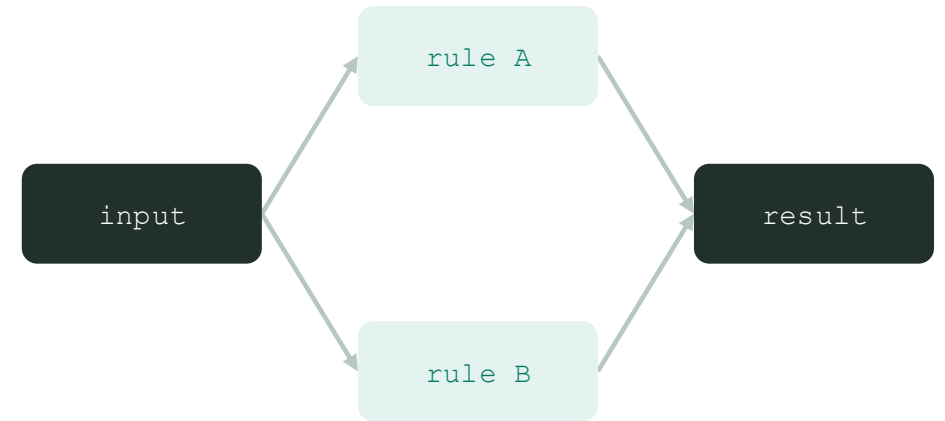
conceptual framework · setup · writing rules · generating a pipeline



Resources



Your questions



rules + files in, a graph of jobs out

Why use a workflow management tool

Doing it by hand

- ✗ Steps live in memory, a README, or scattered scripts
- ✗ Easy to lose track of what ran, with which parameters
- ✗ Re-running means re-running everything from scratch
- ✗ Hard to scale past your own laptop

Sound familiar? If you split your time between the bench and the terminal, you can't also afford to hold the pipeline in your head.



Photo: Eliza Daigle · Pictured: Anup Bhusal

THE MOTIVATION

Why use a workflow management tool

Doing it with Snakemake

- ✓ Rules declare dependencies — order is inferred, not memorized
- ✓ Only what changed gets re-run
- ✓ Same workflow: laptop → cluster → cloud, unchanged
- ✓ Rules + environments become a record of exactly what you did

Same person, same desk, same data — the difference is that the pipeline is now the computer's job to remember.



Photo: Eliza Daigle · Pictured: Anup Bhusal

Today's objectives

By the end of this session, you'll be able to:

-  Explain why a workflow manager pays off for reproducible bioinformatics
-  Read a Snakemake rule and know what each part does
-  Explain how wildcards and file dependencies build a DAG of jobs
-  Run a Snakemake workflow: dry-run, execute, and visualize it
-  Know where to go to learn more



Later this morning: you'll put this into practice directly in the Sourmash lesson.

What is Snakemake?

A Python-based workflow management system for building reproducible, scalable data analyses.



Extends the classic GNU Make idea

rules describe how to turn input files into output files



Plain-text and readable, Python underneath

no bioinformatics or Python experience required to get started



Same workflow, any scale

runs unmodified on a laptop, an HPC cluster, or the cloud

2012

Introduced by Köster & Rahmann as a scalable bioinformatics workflow engine

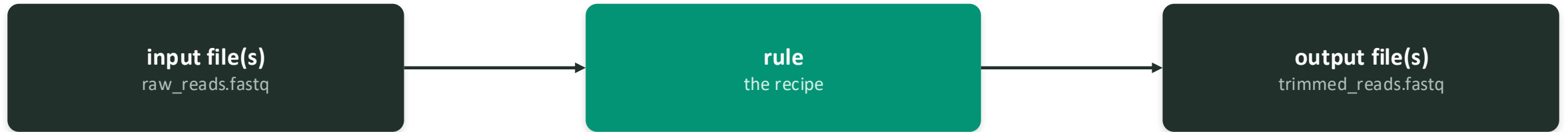
1M+

Downloads on Anaconda.org and still climbing

10+

New papers citing Snakemake every week

A rule: input files in, output files out



Snakemake works backward from the output you ask for
you name the destination; it figures out how to get there



No step numbers to keep straight
order is inferred automatically by matching filenames between rules



Nothing reruns unless it has to
a rule only fires when its output is missing or its input changed

Wildcards: write one rule, apply it to every sample

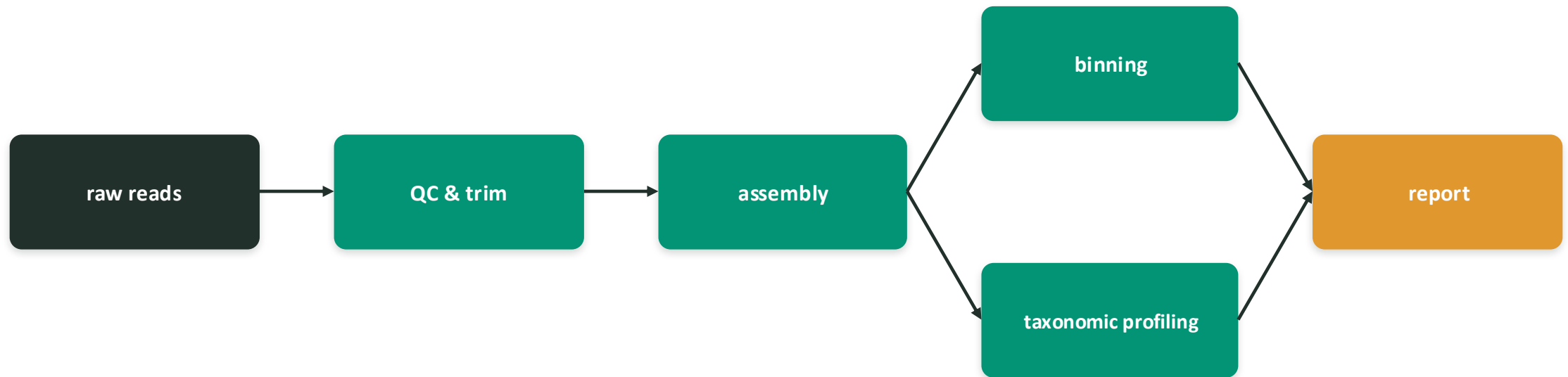


`{sample}` is a placeholder. Snakemake matches it against the files it needs to produce, then runs the rule once per sample.

```
rule trim_reads:
    input: "raw/{sample}.fastq"
    output: "trimmed/{sample}.fastq"
```

The DAG: rules chain together automatically

Output of one rule becomes input to the next — Snakemake assembles a Directed Acyclic Graph (DAG) of jobs, and runs independent branches in parallel.



Binning and profiling don't depend on each other, so Snakemake runs them at the same time. **This is exactly the shape of this afternoon's shotgun-metagenomics pipeline.**

Setup: a standardized project structure

```
.
├── README.md
├── workflow/
│   ├── Snakefile
│   ├── rules/
│   │   ├── qc.smk
│   │   ├── assembly.smk
│   │   └── binning.smk
│   ├── envs/
│   │   └── qc.yaml
│   └── scripts/
├── config/
│   ├── config.yaml
│   └── samples.tsv
├── out/
└── resources/
```



workflow/Snakefile

the entry point that ties all the rules together



rules/*.smk

one file per pipeline stage keeps things readable



envs/*.yaml

a conda environment per rule — exact software versions



config/

sample sheets & run settings, kept separate from logic



out/

outputs only — generated, never hand-edited

Store it in a Git repo with this layout and it's ready to distribute, rerun, and hand off.

Writing rules

```
rule trim_reads:
    input:
        r1 = "raw/{sample}_R1.fastq.gz",
        r2 = "raw/{sample}_R2.fastq.gz"
    output:
        r1 = "trimmed/{sample}_R1.fastq.gz",
        r2 = "trimmed/{sample}_R2.fastq.gz"
    conda:
        "envs/trimmomatic.yaml"
    threads: 4
    shell:
        "trimmomatic PE -threads {threads} "
        "{input.r1} {input.r2} {output.r1} {output.r2}"
```

input:

files that must exist before this rule can run

output:

files this rule promises to create

conda:

the exact software environment for this step

shell:

the command — {input}/{output} auto-fill

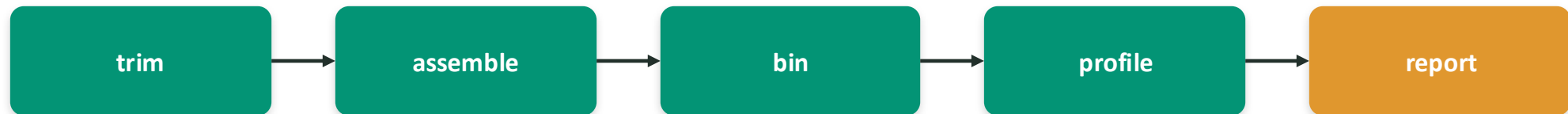
Beyond shell: swap shell: for script: (a .py/.R file) or run: (inline Python) when a step needs more than a command line.

Generating a pipeline

By convention the first rule, `all`, lists the final files you want. Snakemake works backward from there, chaining whichever rules are needed.

```
rule all:  
    input:  
        expand("results/{sample}_report.html", sample=SAMPLES)
```

`expand()` turns a list of sample names into a list of target files — one target per sample, still just one rule.



Ask for the final report — Snakemake figures out that trimming → assembly → binning → profiling has to happen first, in that order.

Running Snakemake

```
snakemake -n
```

Dry run — show the plan, change nothing. Always do this first.

```
snakemake --cores 4
```

Execute the workflow, using up to 4 cores.

```
snakemake --use-conda
```

Auto-create and activate each rule's conda environment.

```
snakemake --cores 1 -p
```

Print the exact shell commands as they run — great for debugging.

```
snakemake --dag | dot -Tpng > dag.png
```

Draw the DAG as an image.



Smart caching: a rule only reruns if its output is missing or its input changed — like make, but for data analysis.

Back to reproducibility



A runnable record

the Snakefile + rules describe exactly what your analysis does, top to bottom



Pinned software

conda: environments capture exact versions — no more “it worked on my machine”



An audit trail

the DAG and logs record what ran, when, and why



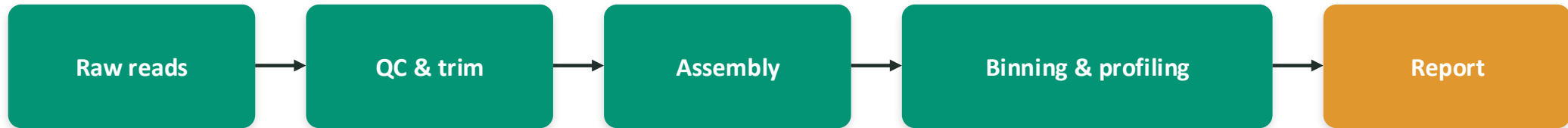
Portable by default

the same workflow moves from your laptop to a cluster like ROAR, unchanged

COMING UP NEXT

This afternoon

In the shotgun data analysis session, you'll run a Snakemake pipeline chaining:



Start with `snakemake -n`

see the full plan before anything runs



Open the rules

they're written to be read, not just executed



Ask questions as you go

that's what we're all here for



All materials: github.com/Penn-State-Microbiome-Center/KickStart-Workshop-2026

Resources



Snakemake documentation

snakemake.readthedocs.io

the full manual — install, syntax, execution, everything



Official tutorial

snakemake.readthedocs.io/en/stable/tutorial

step-by-step walkthrough building a small workflow



Snakemake Workflow Catalog

snakemake.github.io/snakemake-workflow-catalog

browse published, ready-to-adapt pipelines



Carpentries lesson

carpentries-incubator.github.io/snakemake-novice-bioinformatics

free, hands-on Snakemake for Bioinformatics course



Snakemake terminology guide

medium.com/@mandhri.abeysooriya/snakemake-terms-you-will-meet-cheat-sheet

plain-English definitions of every term you'll encounter, with examples



Snakemake Wrappers

snakemake-wrappers.readthedocs.io

ready-made rules for common bioinformatics tools



Questions?

Dr. Jess Grembi

Find me during the break if we run out of time – or ask Eliza, she's run a LOT of Snakemake in the past year!

